

PlomerIA

Plataforma de servicios de plomeria con IA

Informe tecnico y guia de defensa del proyecto

Materia	Desarrollo de Software - 2.o ano
Grupo	Grupo 14
Stack	FastAPI + SQLAlchemy + SQLite (backend) - React 18 + Vite + Zustand (frontend)
Fecha	15/06/2026

1. Resumen ejecutivo

PlomerIA es un marketplace que conecta clientes con plomeros. El cliente describe su problema, una IA (Google Gemini, con un mecanismo de respaldo por palabras clave) lo diagnostica y sugiere hasta 5 profesionales segun cercania y reputacion. El sistema gestiona el ciclo completo del trabajo: solicitud, aceptacion, ejecucion, boleta, finalizacion y calificacion mutua, con chat en tiempo real, moderacion de lenguaje, suspensiones automaticas y un sistema de penalizaciones.

Este informe documenta el trabajo de auditoria y mejora realizado sobre el proyecto: los problemas detectados mediante pruebas manuales, las soluciones aplicadas, su fundamentacion segun los contenidos de la catedra (arquitectura en capas, patrones de diseno, metodologias agiles y aseguramiento de la calidad), y los puntos pendientes. Sirve como guia para exponer y defender el trabajo.

2. Problemas encontrados (pruebas manuales)

La mayoría de los defectos no se descubren leyendo el código sino **usando la aplicacion**. Estos surgieron probando los flujos reales como cliente y como plomero:

#	Problema detectado	Impacto
1	En urgencias no se podía seleccionar al plomero ni enviar la solicitud (funcion indefinida).	Bloqueaba un flujo central.
2	El panel de ganancias no aparecia para plomeros nuevos/ficticios.	El profesional no veia su facturacion.
3	La recomendacion no siempre devolvía 5 profesionales.	Menos opciones para el cliente.
4	Si todos rechazaban, la solicitud quedaba trabada (sin re-buscar ni cerrar).	Flujo sin salida.
5	El calendario mostraba 'proximo lunes' en lugar del dia/horario real.	Turnos mal agendados.
6	Inconsistencia: facturacion con monto pero '0 trabajos' (y viceversa).	Datos contradictorios.
7	Se podía calificar sin cargar la boleta; y se aceptaban montos negativos o en cero.	Calculo de ganancia/comision erroneo.
8	La calificacion habia que buscarla en el historial; incomoda.	Mala experiencia de uso.
9	La cuenta suspendida seguía pudiendo usar la app y loguearse.	La sancion no se cumplía.
10	Duraciones de suspension invertidas (groserias 2 meses / cancelaciones 1 mes).	Reglas de negocio mal aplicadas.
11	El lenguaje ofensivo solo se moderaba en el chat, no al describir el problema.	Falta de control.
12	Se podían elegir turnos en horarios ya pasados (14h siendo las 17:51).	Agenda incoherente.
13	El boton 'Termine el trabajo' se habilitaba antes de tiempo.	Cierre prematuro.
14	Plomeros suspendidos u ocupados aparecian como opcion.	Se asignaba trabajo a quien no podía.
15	Los carteles eran el alert nativo del navegador (estetica ajena a la app).	Imagen poco profesional.

3. Mejoras realizadas

Cada problema se resolvió y, además, se reforzaron reglas de negocio y la consistencia general:

Area	Mejora aplicada
Solicitudes	Envío simultáneo a hasta 5 plomeros; al aceptar uno, a los demás les desaparece al instante; la dirección del cliente queda oculta hasta que un profesional acepta.
Tiempos de trabajo	'En camino' solo desde 2h antes del turno; 'Finalizar' solo 2h después de marcar en camino (botón deshabilitado hasta entonces); no se agendan turnos en horarios pasados.
Disponibilidad	Un plomero con trabajo activo no aparece para otros; al finalizar vuelve a estar disponible desde la hora próxima; vencimiento del tiempo de respuesta (3h / 30min, con extensión al día siguiente si se envía pasadas las 22h).
Calificación	Aparece al finalizar (en 'En curso' / 'Mis solicitudes'); al calificar, el trabajo pasa al historial; 5 estrellas automáticas si no se califica en 48h.
Ganancias	El monto entra a ganancias al finalizar (con boleta obligatoria); el conteo de trabajos y la puntuación se actualizan al calificar.
Recomendación	Siempre intenta 5; orden por cercanía, luego puntuación y, como desempate, coincidencia de oficio; excluye suspendidos y ocupados.
Moderación	Filtro de groserías en chat y en la descripción del problema; a la 3a falta, suspensión automática con cartel y cierre de sesión.
Suspensiones	Groserías: 1 mes; 3 cancelaciones consecutivas: 2 meses; bloqueo total (login, recuperar clave, uso) y reactivación automática al vencer el plazo.
Interfaz	Carteles propios con la estética de la app y color según el tipo (éxito, error, aviso, info); nombre del usuario visible y consistente.
Calidad de código	Eliminación de prints de depuración, archivos y carpetas sin uso; suite de pruebas unitarias con pytest (46 pruebas).

4. Fundamentacion segun el material de la catedra

Las decisiones no son arbitrarias: responden a principios de Ingenieria de Software vistos en clase.

Principio / contenido	Como se aplico en PlomerIA
Separacion de responsabilidades (SoC)	El backend se divide en capas (routers, services, repositories, models). Cada una tiene una unica razon de cambio.
Responsabilidad unica (SRP - SOLID)	La moderacion de lenguaje, las suspensiones, las calificaciones y las solicitudes viven en modulos separados.
Bajo acoplamiento / alta cohesion	Los servicios no acceden a la base directamente: lo hacen a traves de repositorios, por lo que cambiar el motor de datos no afecta la logica.
Reglas de negocio en el dominio	Plazos, penalizaciones y suspensiones estan centralizados en servicios, no dispersos en la interfaz.
Validacion y robustez	La boleta es obligatoria para finalizar, se rechazan montos invalidos y el lenguaje ofensivo se filtra: el sistema no confia ciegamente en la entrada.
Testeo de caja blanca	Las reglas dificiles de verificar a mano (orden de recomendacion, calculo de fechas, penalizaciones) se aislaron en funciones puras y se probaron con pytest.

5. Arquitecturas utilizadas

El proyecto combina varios estilos arquitectonicos vistos en la materia:

Arquitectura / estilo	Donde y para que
Cliente-Servidor	El frontend (cliente, React) consume al backend (servidor, FastAPI). Responsabilidades separadas y desplegadas por separado.
Arquitectura en capas (Layered)	Backend organizado en capas: Presentacion (routers/API) -> Logica de negocio (services) -> Acceso a datos (repositories) -> Modelos/Persistencia (SQLAlchemy).
API REST	Comunicacion HTTP con recursos (solicitudes, plomeros, calificaciones...). Sin estado, facil de probar y escalar.
WebSocket (tiempo real)	El chat usa una conexion persistente bidireccional para mensajes instantaneos, en vez de consultar el servidor cada pocos segundos.
SPA (Single Page Application)	El frontend es una unica pagina que cambia de vista sin recargar (React + Vite), con manejo de estado global (Zustand).
ORM (Object-Relational Mapping)	SQLAlchemy mapea clases de Python a tablas SQLite: se trabaja con objetos, no con SQL escrito a mano.
Tarea programada (scheduler)	Un proceso en segundo plano corre cada hora: cierra calificaciones vencidas (5 estrellas a las 48h) y solicitudes sin respuesta.

Conteo real: 10 routers, 17 servicios y 11 repositorios - evidencia de la separacion en capas.

6. Patrones de diseno utilizados

Un patron es una solucion probada a un problema recurrente. Estos se identifican en el codigo:

Patron	Donde se usa	Por que es mejor que la alternativa	Beneficio
Repository	Carpeta repositories/ (uno por entidad).	Alternativa: poner consultas SQL dentro de la logica de negocio. Eso mezcla responsabilidades y ata el codigo a SQLite.	Aisla el acceso a datos; se puede cambiar de base sin tocar la logica; facilita pruebas con datos simulados.
Service Layer / Fachada	Carpeta services/.	Alternativa: logica de negocio en los controladores (routers). Los routers se vuelven enormes y no reutilizables.	Centraliza las reglas; los routers quedan finos; la misma regla sirve para REST y WebSocket.
Singleton	ConnectionManager del chat (una sola instancia 'manager').	Alternativa: crear un gestor por cada pedido. Se perderia el registro de quien esta conectado.	Un unico punto que conoce todas las salas de chat activas y reparte los mensajes.
Strategy + Fallback	ia_service: Gemini y, si falla, diagnostico por palabras clave.	Alternativa: depender solo de la IA. Si la API cae o no hay clave, la app deja de funcionar.	El sistema sigue operativo aunque la IA externa falle; la estrategia se elige en tiempo de ejecucion.
DTO (esquemas)	Esquemas Pydantic (schemas/).	Alternativa: devolver objetos de la base directamente. Expone datos internos y no valida la entrada.	Valida y da forma a los datos de entrada/salida; contrato claro de la API.
Funciones puras	Orden de recomendacion y calculo de fechas.	Alternativa: mezclar la logica con accesos a base y red. Se vuelve imposible de probar de forma aislada.	Mismos parametros, mismo resultado: se prueban con pytest sin levantar la base.

7. Metodologías ágiles vs. cascada

El trabajo se desarrollo de forma **ágil**: en ciclos cortos e iterativos. Se probaba la app, se detectaba un problema, se corregia, se volvía a probar y se ajustaba según el feedback. La usuaria cumplió el rol de Product Owner y de tester, validando cada incremento.

Aspecto	Ágil (lo que usamos)	Cascada (si la hubieramos usado)
Entregas	Incrementales: funcionalidad usable en cada ciclo.	Una sola entrega al final del proyecto.
Cambios	Bienvenidos; se reordenan prioridades sobre la marcha.	Difíciles y costosos una vez fijado el plan.
Errores	Se detectan temprano al probar cada incremento.	Aparecen recién en la fase de prueba, al final.
Feedback	Continuo (la usuaria probaba y pedía ajustes).	Solo al cierre; alto riesgo de construir algo equivocado.
Riesgo	Bajo y distribuido.	Concentrado al final; si algo está mal, se rehace mucho.

Que habría pasado con cascada? Habríamos definido todos los requisitos al inicio y construido todo de corrido para probar recién al final. Defectos como los del calendario, las suspensiones o los tiempos de trabajo se habrían descubierto tarde, cuando corregirlos cuesta mucho más. El método ágil permitió encontrarlos y resolverlos en el momento.

8. El impacto de no tener un Scrum Master

Scrum define tres roles: Product Owner (que construir y con que prioridad), Equipo de Desarrollo (lo construye) y Scrum Master (facilita el proceso, remueve impedimentos y protege al equipo). En este trabajo no hubo un Scrum Master, y eso se noto:

Funcion del Scrum Master	Consecuencia de su ausencia
Remover impedimentos	Problemas de entorno (fallas al guardar archivos, conflictos de Git) frenaron el avance y hubo que resolverlos sin guía de proceso.
Facilitar y ordenar el trabajo	La priorización quedo a cargo de la misma persona que probaba y decidía; se mezclaron roles.
Proteger el foco del equipo	Sin alguien que cuidara el alcance, surgieron retrabajos y cambios de rumbo a mitad de camino.
Asegurar la cadencia	Sin ceremonias (daily, review, retro), el seguimiento del avance fue informal.

9. QA (Aseguramiento de la Calidad)

QA significa *Quality Assurance* (Aseguramiento de la Calidad). Es el conjunto de actividades que buscan que el software cumpla lo que debe y no falle: no es solo 'probar al final', sino cuidar la calidad durante todo el desarrollo. En este proyecto el QA tuvo dos frentes:

Tipo de prueba	Que se hizo	Herramienta
Pruebas unitarias (caja blanca)	Se verificaron reglas criticas: orden de recomendacion, calculo de fechas/turnos, penalizaciones, franjas horarias, oficios, moderacion y boleta. 46 pruebas que pasan.	pytest
Pruebas manuales (caja negra / exploratorias)	Uso real de la app como cliente y como plomero para detectar los problemas listados en la seccion 2.	La propia aplicacion
Validaciones en el sistema	Boleta obligatoria, rechazo de montos invalidos, control de estados y de tiempos: el software se defiende de entradas incorrectas.	Reglas en los servicios

Las pruebas unitarias dan una red de seguridad: si una mejora rompe una regla ya probada, pytest lo avisa al instante, antes de que llegue al usuario.

10. Integracion back-front y puesta en marcha

Para mostrar la app funcionando se ejecutan dos procesos que se comunican entre si:

Parte	Como se ejecuta	Detalle
Backend (FastAPI)	uvicorn (servidor) en el puerto 8000.	Expone la API REST y el WebSocket del chat.
Frontend (React)	Vite (servidor de desarrollo) en el puerto 5173.	Es lo que se ve en el navegador (localhost:5173).
Conexion entre ambos	El front llama al back por HTTP (Axios) y WebSocket; CORS habilita esa comunicacion.	Por eso, para la demo, se levantan los dos a la vez.

Conceptos para la defensa:

Termino	Que significa	Estado en el proyecto
Deployar (deploy)	Publicar la aplicacion en un servidor accesible por internet, para que cualquier usuario la use sin tener el codigo.	Aun no desplegado: la demo corre en localhost (la propia computadora).
Docker	Empaqueta la app y todo lo que necesita (Python, librerias, configuracion) en un 'contenedor' que corre igual en cualquier maquina. Sirve para desplegar y evita el clasico 'en mi compu anda'.	No esta en esta rama; se estuvo probando aparte.
localhost	La propia maquina actuando como servidor (para desarrollo y demostracion).	Es donde se muestra la app hoy.

11. La IA como herramienta y el trabajo humano

La Inteligencia Artificial es una herramienta poderosa que acelera tareas: escribir código repetitivo, encontrar errores, sugerir soluciones. Pero **no reemplaza el criterio humano**. A lo largo del proyecto, la IA propuso y ejecuto cambios, pero las decisiones de fondo fueron humanas:

Lo que aporta la IA	Lo que sigue siendo humano
Velocidad para escribir y corregir código.	Definir las reglas de negocio (plazos, penalizaciones, suspensiones).
Detectar patrones y posibles errores.	Probar la app de verdad y notar que 'se siente mal' para el usuario.
Sugerir alternativas técnicas.	Elegir entre esas alternativas con criterio y según el contexto.
Generar documentación y pruebas.	Validar que lo generado sea correcto y tenga sentido.

Además, aun con IA, **un proyecto lleva tiempo real**: entender el problema, probar, encontrar defectos sutiles, decidir como resolverlos, verificar que no se rompa otra cosa y dejar todo coherente. El pensamiento crítico y el razonamiento personal son insustituibles: la IA ejecuta, pero la persona dirige, cuestiona y se hace responsable del resultado.

12. Pendientes: que queda por corregir y mejorar

Pendiente	Descripción
Vencimiento de respuesta fino	El scheduler corre cada hora; una urgencia de 30 min podría cerrarse con algo de demora. Se podría reducir el intervalo.
Refactor de servicios grandes	solicitud_service concentra mucha logica; convendría dividirlo (solicitudes / reasignacion / boleta) para respetar mejor SRP.
Pruebas de integracion	Sumar pruebas que recorran la maquina de estados completa (aceptar, cancelar en estado invalido, finalizar sin boleta) además de las unitarias.
Integracion continua (CI)	Configurar que las pruebas corran automaticamente en cada push (por ejemplo, GitHub Actions).
Contenerizacion y deploy	Terminar la prueba con Docker y desplegar la app en un servidor para acceso real.
Avisos en tiempo real de suspension	Hoy el cierre de sesion por suspension se dispara al hacer una accion; podría ser instantaneo via WebSocket.
Limpieza de advertencias	Actualizar usos deprecados de SQLAlchemy/Pydantic que aparecen como warnings.

13. Conclusion

PlomerIA es un sistema completo que aplica conceptos centrales de la materia: arquitectura en capas, patrones de diseño, separación de responsabilidades, aseguramiento de la calidad con pruebas y un desarrollo guiado por una metodología ágil. El trabajo de auditoria manual permitio encontrar y corregir defectos que solo se ven usando la aplicacion, y reforzar las reglas de negocio para que el sistema sea coherente y robusto. Quedan mejoras por delante (deploy, CI, refactors), propias de un producto que sigue evolucionando.